

Connect to the conference wi-fi:

Network: dbt Summit

Passcode: run_dbt!

Once you're seated, please join the WiFi – this
helps everyone connect smoothly!

**Access AI closed captions and
translation on your personal device.**



Accelerating Analytics with AI



Lucas Silbernagel

Resident Architect
dbt Labs



Faith McKenna

Manager, Technical Instruction
dbt Labs

Agenda

01

**Safe-acceleration
mindset**

02

**An agent for
Analytics Engineering**

03

Wizard in action

04

Reviewing agentic work

05

Capture your standards

Housekeeping 🖋️

- Questions? Raise your hand!
- Product feedback→Expert Lounge
- Slides are available post-workshop.
- Workshop handout: [link](#)
- Demo account→deactivated in 2 days.
- Take the post-workshop [survey](#)!



Sandbox Setup

Workshop Registration for dbt Platform

Log out of any dbt accounts

- or use private/incognito session

Workshop Registration page:
workshops.us1.dbt.com/workshop

- Select **Accelerating Analytics with AI**
- Enter the Passcode: **Summit2026!**
 - **Take note:** User ID and Password
 - Take a picture or screenshot



Workshop Signup

Please provide your first name, last name, and which workshop you will be attending. After submitting this form, you may log into the new account for your workshop.

First Name

Last Name

Workshop

Passcode

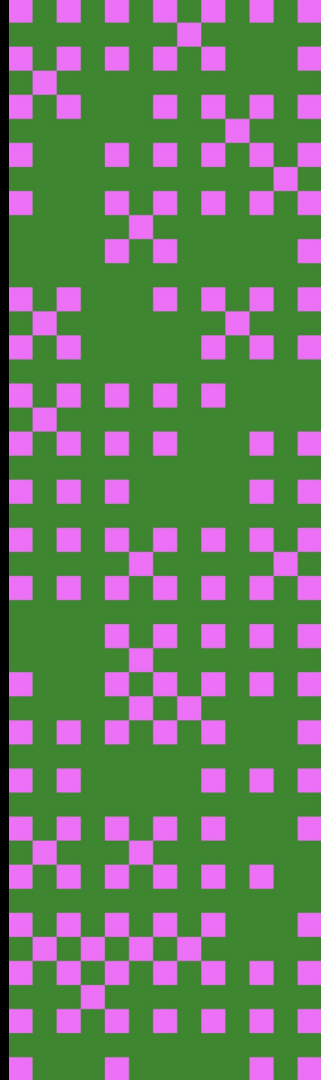
☐ I agree to dbt's [Terms of Service](#)



Project setup

Let's head to **Orchestration** > **Jobs** > **Prod job** in dbt Platform

And **run** the job!





Safe-acceleration mindset

AI introduces code issues

Liu et al. (March, 2026) analyzed **304,362** AI-authored commits across **6,275** GitHub repositories (100+ stars, Python/JS/TS)

484,606

distinct code issues

61.2%

of repos had code issues

89.1%

of code issues are code smells

Fast is not right

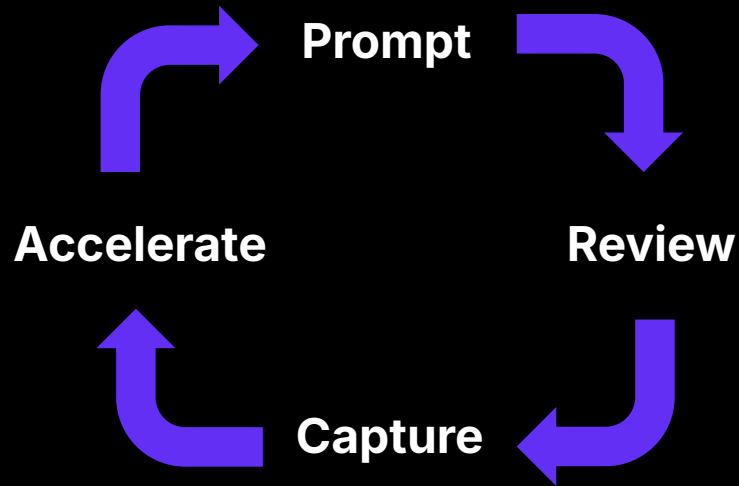
AI can build dbt code fast.

But the output will not automatically match your conventions.

Speed without standards creates rework and review friction.

Move faster and keep engineering quality high.

The loop





An agent for Analytics Engineering

dbt Copilot → dbt Wizard

How AI usage in dbt evolved

Copilot

Task-specific assistance for creating docs, tests, semantic models, and SQL.



Wizard

A dbt-aware development agent that can **investigate** context, make coordinated project changes, **validate** results, and help **ship** governed dbt work.



dbt Wizard

AI agents are only as good as the context they have available.

dbt Wizard has an in-built context layer that lets it understand your data.

How!?

Wizard's superpowers

01

dbt CLI + Project files

02

dbt Agent Skills

03

dbt MCP Server

No configuration, no context feeding. Natively grounded in your dbt project.

dbt CLI + Project files

Wizard can access your project's file tree.
It can also run most dbt CLI commands.

Project files

- Read
- Edit
- Create
- Delete

CLI

- parse/compile
- build/run
- test
- run-operation
- show
- ...

dbt Agent Skills

Skills that help AI agents understand and execute dbt workflows more effectively.

What's included?

- **Analytics engineering:** Build and modify dbt models, write tests, explore data sources
- **Semantic layer:** Create metrics, dimensions, and semantic models with MetricFlow
- **dbt Mesh:** Work with multi-project setups, cross-project refs, model governance (contracts, versions, access)
- **Platform operations:** Troubleshoot job failures, configure the dbt MCP server
- **Migration**

dbt MCP Server

Tools that allow your agent to interact with your dbt project.

What's included?

- **Semantic Layer**
 - list_metrics
 - get_entities
 - query_metrics
 - ...
- **Discovery**
 - get_model_details
 - get_lineage
 - ...
- **Product Docs**
 - get_product_doc_pages
 - Search_product_docs
 - ...
- **Admin API**
 - list_jobs
 - trigger_job_run
 - ...
- **dbt Codegen**
 - generate_model_yaml
 - ...
- **dbt LSP**
 - get_column_lineage
 - ...

Where to use Wizard

GA

01

dbt Studio

*Public
Beta*

02

CLI

*Private
Preview*

03

Desktop app



Wizard in action

dbt Wizard Use Cases

Understand an existing dbt project

"What does this model do?" → Translates SQL logic into natural language

"What tests exist for this part of the DAG?"

↳ Has project metadata awareness

"Where is this column used downstream?"

↳ Accesses column-level lineage

dbt Wizard Use Cases

Generate and refactor dbt models

"Standardize CTE structure."



Performs multi-file edits

"Convert legacy SQL into dbt models."



Knows dbt best practices + creates new files

"Create staging models from raw sources."



Infers source schema from target + Codegen

dbt Wizard Use Cases

Debugging dbt

"Why is this job failing?" → Pulls job information and reads errors

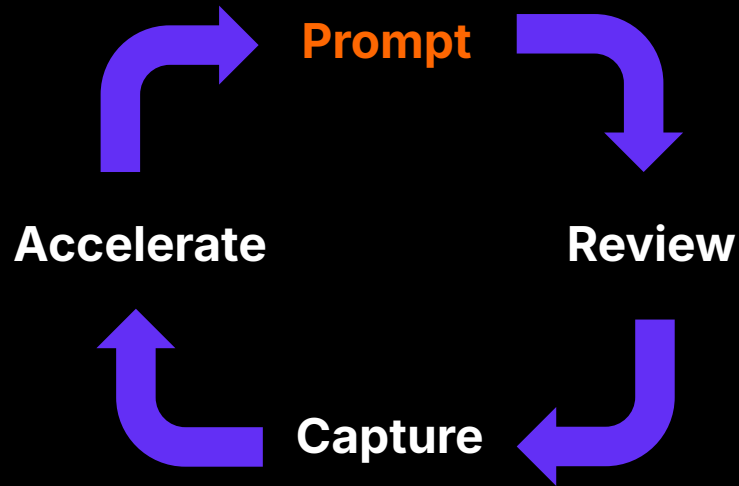
"Why is this model failing?"



Reads and troubleshoots CLI errors

"Why is my project not compiling?"

The loop



Practical block #1

A new mart model

Read `exercises/exercise1.md` in your project for details.

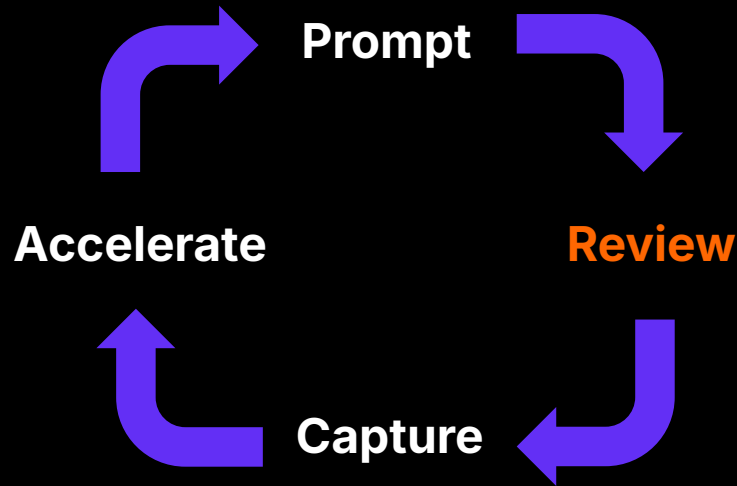
05:00



Reviewing agentic work

The loop

Subheader to support the main title



What to review with Wizard

Wizard asks to parse and build the changes.

- dbt parse
- dbt build

Wizard can preview the data in chat when prompted.

- dbt show
- Can create simple visuals

Wizard can show the model in the DAG when prompted.

- Lineage
- Column-level lineage

What to review yourself

dbt Project requirements

- Owner/tag/contract/etc. required?
- SQL style
- Materializations
- No join fan-out
- Naming conventions

Business requirements

- Filters
- Timezones
- Fiscal calendar
- etc.

Wizard infers standards from surrounding files but there can be inconsistencies

Practical block #2

Review the changes.

Read `exercises/exercise2.md` in your project for details.

05:00

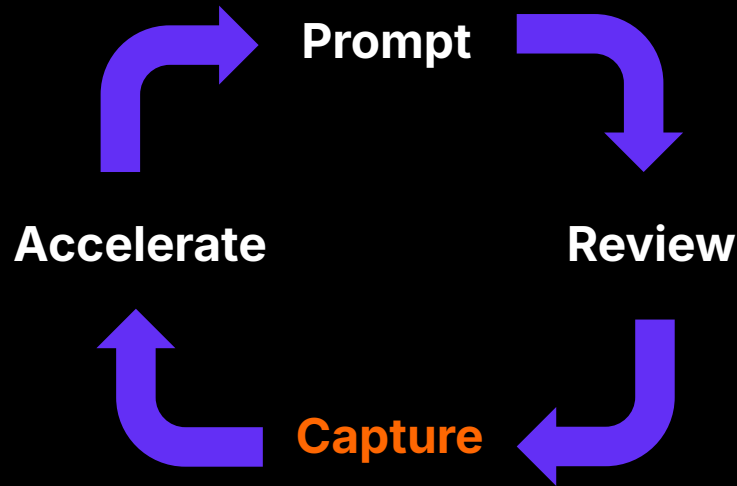
05:00



Capture your standards

The loop

Subheader to support the main title



How to capture recurring context?

Skills

- Reusable, task-specific instruction packages that tell an agent how to perform a certain kind of work
- Automatically used when the agent sees a match with a skill and the task at hand
- Can be explicitly invoked in the prompt
- Auto-discovered in `.claude/skills/NAME/SKILL.md`
- Recommended structure



```
.agents/  
  skills/  
    your-skill-name/  
      SKILL.md
```

How to capture recurring context?

AGENTS.md

- Instruction file that tells an agent how to behave or operate within a specific folder, project, module, or context
- Always-on context for any prompt
- Wizard reads `AGENTS.md` files stored at the project root
- Also reads `AGENTS.override.md`, `CLAUDE.md`, `.claude/CLAUDE.md`

Capture requirements

Project-specific requirements

- Owner/tag/contract/etc. required?
- SQL style
- Incremental config
- No join fan-out
- Naming conventions



create-mart-model skill

Business-specific requirements

- Filters
- Timezones
- Fiscal calendar
- etc.



AGENTS.md

Practical block #3

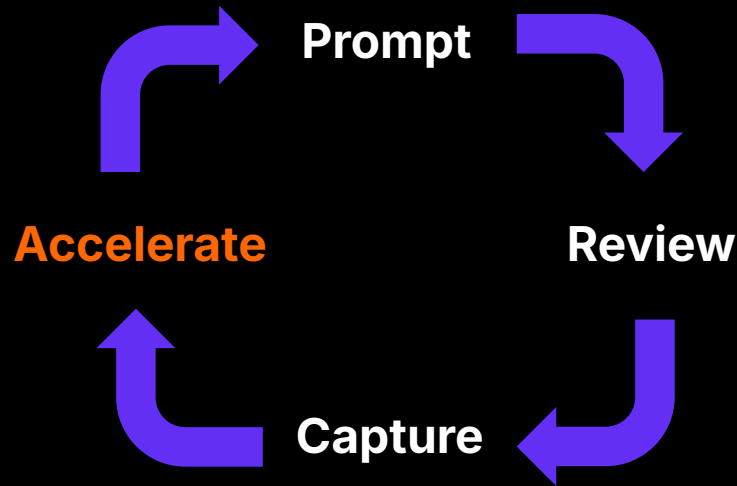
Turn one-off corrections into
reusables tools.

Read `exercises/exercise3.md` in your project for details.

05:00

The loop

Subheader to support the main title



Practical block #4

See the payoff.

Read `exercises/exercise4.md` in your project for details.

05:00



Putting it all together

Practical block #5

Let's do the whole cycle again.

- New task
- Start with an initial prompt
- Review the output
- Capture corrections in reusable tools when applicable
- Delete the files and run the prompt again

05:00

Thank you!



Feedback 🙏

<https://dbtlearn.typeform.com/dbt-summit-labs>

or scan QR code.

